

ENTERPRISE AUTOMATION FRAMEWORK SPECIFICATION

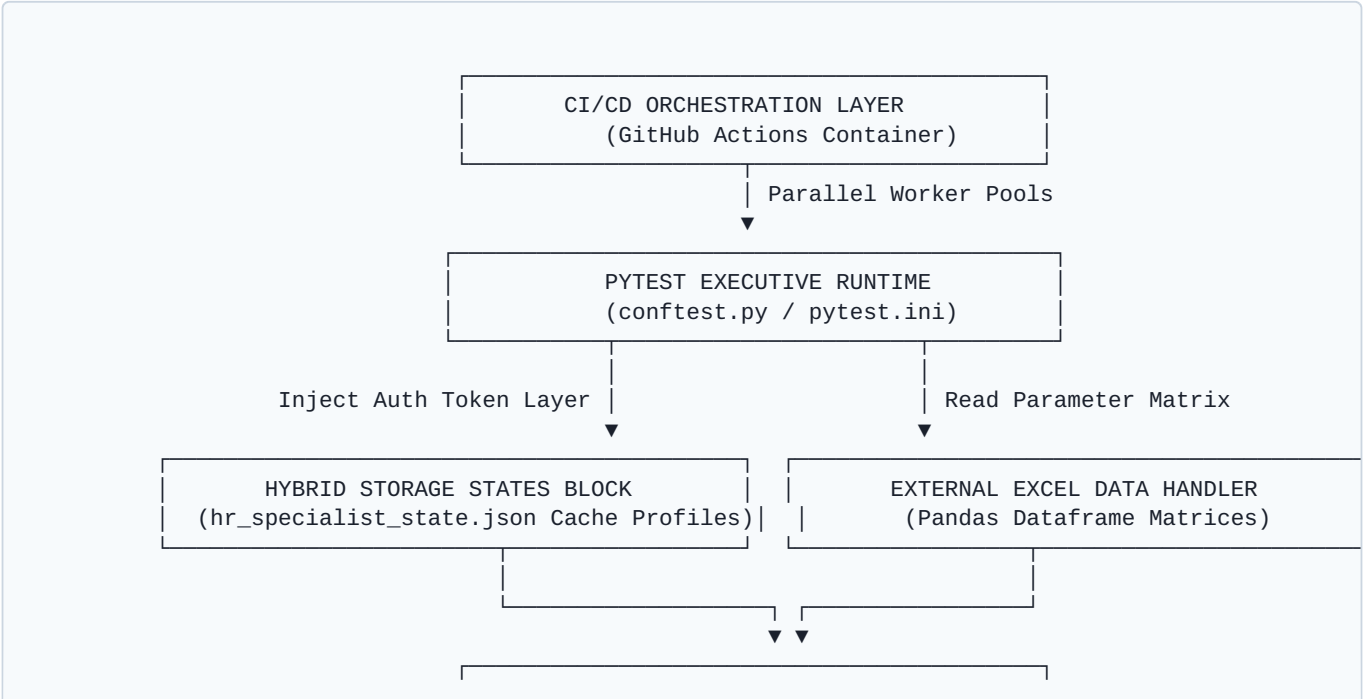
Robust Playwright Architecture Designed for Oracle HRMS Ecosystems

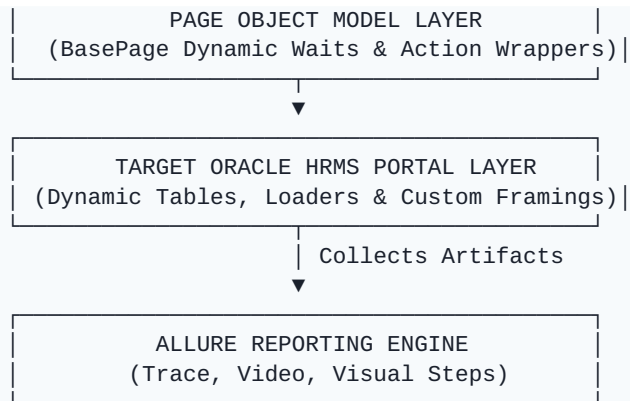
Target Platform	Oracle HRMS Suite (Core HR, Payroll, Performance, Onboarding)	Date Engineered	May 16, 2026
Core Engine Stack	Python 3.11 / Playwright / Pytest / Allure Reporting Matrix	Orchestration	GitHub Actions Container Matrix (Parallel Core Workers)

1. Executive Architectural Blueprint

This architecture specification defines the enterprise automation ecosystem tailored specifically for complex, highly dynamic **Oracle HRMS** target platforms. Given Oracle's asynchronous loading behaviors, dynamic element DOM mapping, and complex business data configurations, this framework guarantees extreme stability, maintainability, and rapid execution scales via:

- **Hybrid Authentication Lifecycle Engine:** Serializes pre-authenticated worker session tokens safely into state vectors to eliminate redundant UI logins, while providing fallback UI verification tests for security auditing.
- **External Data Separation Engine:** Powered by Pandas and openpyxl to decouple testing logical variables entirely from execution paths via isolated spreadsheets.
- **Scalable Thread Management:** Harnesses multi-threaded concurrent isolation inside parallel Linux environments.





2. System Repository Hierarchy

The blueprint implements a strict segregation of concerns ensuring configurations, test parameters, scripts, helpers, and assets live inside structural isolation:

```

oracle-hrms-playwright/
├── .github/workflows/playwright-ci.yml # CI/CD production deployment workflow
├── config/
│   ├── env_config.py # Global structural profile environment loaders
│   ├── stage_config.json # Staging application instance parameters
│   └── uat_config.json # User Acceptance Testing validation properties
├── src/
│   ├── locators/
│   │   ├── login_elements.py # Interface identifiers mapping for SSO portals
│   │   ├── core_hrms_elements.py # Universal navigation bar / wizard identifiers
│   │   └── employee_mgmt_elements.py # Core HR data input element descriptors
│   ├── page_objects/
│   │   ├── base_page.py # Safe element wait actions wrapping core engines
│   │   ├── login_page.py # Standard sequential interface login steps
│   │   └── employee_page.py # Employee transactional page wizards
│   └── utils/
│       ├── excel_handler.py # Robust dataframe matrix spreadsheet parser
│       └── data_reader.py # Dynamic profile metadata environment hooks
├── storage_states/ # Pre-compiled multi-role serialized auth dumps
├── test_data/
│   ├── staging/user_records.xlsx # Staging verification matrix datasets
│   └── uat/user_records.xlsx # User Acceptance operational scenario blocks
├── tests/
│   ├── auth_suites/test_login_flows.py # Standard UI security pathway validations
│   ├── core_hr/test_employee_onboarding.py # Parametric parallel regression tests
│   └── conftest.py # Centralized fixture architectures
  
```

3. Structural Implementation Guide

Step 1: Foundational Framework Infrastructure & Config Profiles

Establishing runtime modules, dependencies, core executing paths, and strict parameter checking using Pydantic.

```
# requirements.txt
playwright==1.42.0
pytest==8.1.1
pytest-playwright==0.4.4
pytest-xdist==3.5.0
allure-pytest==2.13.2
pandas==2.2.1
openpyxl==3.1.2
pydantic==2.6.4
```

```
# pytest.ini
[pytest]
addopts = --v --tb=short --alluredir=allure-results --clean-alluredir -n auto
testpaths = tests
markers =
    smoke: Vital path verification for quick environment health checks
    regression: Core end-to-end Oracle HRMS business transactions
    auth: Dedicated validation suites targeting authentication/SSO modules
```

```
# config/env_config.py
import os
import json
from pathlib import Path
from pydantic import BaseModel

BASE_DIR = Path(__file__).resolve().parent.parent

class EnvironmentConfig(BaseModel):
    base_url: str
    api_url: str
    default_timeout: int = 30000

def load_env_configuration() -> EnvironmentConfig:
    target_env = os.getenv("TARGET_ENV", "stage").lower()
    config_path = BASE_DIR / "config" / f"{target_env}_config.json"
    if not config_path.exists():
        raise FileNotFoundError(f"Configuration profile not found for target: {target_env}")
    with open(config_path, "r") as file:
        return EnvironmentConfig(**json.load(file))

current_config = load_env_configuration()
```

Step 2: Base Action Wrapper Engine & Enterprise Locators Matrix

Intercepts actions to inject stability wrappers that target Oracle UI latency anomalies like AJAX spin overlays and screen transitions.

```

# src/page_objects/base_page.py
from playwright.sync_api import Page
import logging

logger = logging.getLogger(__name__)

class BasePage:
    def __init__(self, page: Page):
        self.page = page
        self.timeout = 30000

    def navigate(self, url: str):
        logger.info(f"Navigating to destination: {url}")
        self.page.goto(url, wait_until="networkidle")

    def click_element(self, selector: str, timeout: float = None):
        t = timeout if timeout else self.timeout
        locator = self.page.locator(selector)
        locator.wait_for(state="visible", timeout=t)
        locator.scroll_into_view_if_needed()
        locator.click()

    def fill_input(self, selector: str, value: str, timeout: float = None):
        t = timeout if timeout else self.timeout
        locator = self.page.locator(selector)
        locator.wait_for(state="visible", timeout=t)
        locator.fill(value)

    def wait_for_oracle_spinner_to_disappear(self, spinner_selector: str, timeout: float = 60000):
        try:
            self.page.locator(spinner_selector).wait_for(state="detached", timeout=timeout)
        except Exception as e:
            logger.warning(f"Oracle loading spinner block did not clear: {str(e)}")

```

```

# src/locators/login_elements.py
class LoginElements:
    USERNAME_FIELD = "input[name='ssouusername']"
    PASSWORD_FIELD = "input[name='password']"
    LOGIN_SUBMIT_BTN = "button#submit_btn"
    SSO_INDICATOR = "div.sso-badge"

# src/locators/employee_mgmt_elements.py
class EmployeeMgmtElements:
    NAV_MENU_BAR = "button[aria-label='Main Navigation']"
    EMPLOYEE_SEARCH_INPUT = "input#emp_search_query"
    SUBMIT_TRANSACTION_BTN = "button.oracle-action-submit"
    ORACLE_BUSY_SPINNER = "div.process-loading-overlay"

```

Step 3: Multi-Role Hybrid Authentication & Driver Lifecycles

Orchestrates session state caching to bypass login panels on standard threads, boosting runtime performance.

```
# tests/conftest.py
import os
import pytest
from pathlib import Path
from playwright.sync_api import sync_playwright, BrowserContext, Page
from config.env_config import current_config
from src.page_objects.login_page import LoginPage

STORAGE_DIR = Path(__file__).resolve().parent.parent / "storage_states"

@pytest.fixture(scope="session", autouse=True)
def generate_global_auth_states():
    STORAGE_DIR.mkdir(exist_ok=True)
    force_refresh = os.getenv("REFRESH_AUTH", "false").lower() == "true"
    hr_specialist_file = STORAGE_DIR / "hr_specialist_state.json"

    if force_refresh or not hr_specialist_file.exists():
        with sync_playwright() as p:
            browser = p.chromium.launch(headless=True)
            context = browser.new_context()
            page = context.new_page()
            login_flow = LoginPage(page)
            page.goto(current_config.base_url)
            login_flow.execute_system_login("HR_SPEC_USER", "SecurePassword123")
            context.storage_state(path=str(hr_specialist_file))
            browser.close()

@pytest.fixture(scope="function")
def authenticated_hr_context() -> BrowserContext:
    state_file = STORAGE_DIR / "hr_specialist_state.json"
    with sync_playwright() as p:
        browser = p.chromium.launch(headless=True)
        context = browser.new_context(storage_state=str(state_file), viewport={"width": 1440, "height": 900})
        yield context
        context.close()

@pytest.fixture(scope="function")
def hr_page(authenticated_hr_context) -> Page:
    page = authenticated_hr_context.new_page()
    yield page
    page.close()
```

Step 4: Pandas Parameterization Engine Interface

Interacts with externalized structures, formatting grid data points smoothly into isolated dynamic execution parameters.

```
# src/utils/excel_handler.py
import pandas as pd
from pathlib import Path
import logging

logger = logging.getLogger(__name__)

class ExcelHandler:
    @staticmethod
    def read_test_dataset(file_relative_path: str, sheet_name: str) -> list[dict]:
        base_dir = Path(__file__).resolve().parent.parent.parent
        full_path = base_dir / "test_data" / file_relative_path
        if not full_path.exists():
            raise FileNotFoundError(f"Missing data source file: {full_path}")
        try:
            df = pd.read_excel(str(full_path), sheet_name=sheet_name)
            return df.fillna("").to_dict(orient="records")
        except Exception as e:
            logger.error(f"Engine failure reading data sheets: {str(e)}")
            raise e
```

Step 5: Operational Testing Suites & Trace Assertions

Provides explicit tracking steps linked cleanly to automated test validation reporting logic.

```
# tests/core_hr/test_employee_onboarding.py
import pytest
import allure
from src.utils.excel_handler import ExcelHandler
from src.page_objects.employee_page import EmployeePage
from config.env_config import current_config

test_scenarios = ExcelHandler.read_test_dataset("staging/user_records.xlsx", sheet_name="Onboarding")

@allure.epic("Oracle HRMS Portal")
@allure.feature("Core HR Transactions")
@pytest.mark.regression
@pytest.mark.parametrize("scenario", test_scenarios)
def test_parameterized_employee_onboarding(hr_page, scenario):
    employee_page = EmployeePage(hr_page)
    allure.dynamic.title(f"Onboarding Run: {scenario['ScenarioID']} - {scenario['EmpFirstName']}")

    with allure.step("Navigate to Employee Setup Interface"):
        hr_page.goto(f"{current_config.base_url}/OA_HTML/AppsLocalHome.jsp")
        employee_page.navigate_to_hiring_wizard()

    with allure.step(f"Populate profile data for: {scenario['EmpFirstName']}"):
        employee_page.fill_demographics(scenario['EmpFirstName'], "Rana")

    with allure.step(f"Assign workplace elements: {scenario['JobCode']}"):
        employee_page.assign_job_and_salary(scenario['JobCode'], scenario['SalaryGrade'])

    with allure.step("Commit worker assignment to Oracle engine"):
        employee_page.submit_assignment_transaction()

    with allure.step("Verify transactional status message"):
        success_message = employee_page.get_transaction_status_message()
        assert "Transaction Approved" in success_message
```

Step 6: Production-Class CI/CD Multi-Worker Pipeline Configuration

GitHub Actions definitions allocating isolated worker resources for running tests and publishing reporting dashboard history pages.

```
# .github/workflows/playwright-ci.yml
name: Oracle HRMS Playwright Automation Pipeline
on:
  push:
    branches: [ main ]
  schedule:
    - cron: '0 2 * * *'
jobs:
  regression-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'
          cache: 'pip'
      - run: pip install -r requirements.txt
      - run: playwright install chromium --with-deps
      - run: pytest
        continue-on-error: true
      - uses: simple-elf/allure-report-action@master
        if: always()
        with:
          allure_results: allure-results
      - uses: peaceiris/actions-gh-pages@v3
        if: always()
        with:
          github_token: ${ secrets.GITHUB_TOKEN }
          publish_dir: ./allure-report
```

Prepared by: Azmat

Designation: Automation Architect

Status: Approved Architecture Specification Blueprint